

STOMPC

Stochastic Model-Predictive Control with Uppaal Stratego

Goorden, Martijn; Jensen, Peter Gjør; Larsen, Kim Guldstrand; Samusev, Mihail; Srba, Jiri; Zhao, Guohan

Published in:

International Symposium on Automated Technology for Verification and Analysis

DOI (link to publication from Publisher):

[10.1007/978-3-031-19992-9_21](https://doi.org/10.1007/978-3-031-19992-9_21)

Publication date:

2022

Document Version

Accepted author manuscript, peer reviewed version

[Link to publication from Aalborg University](#)

Citation for published version (APA):

Goorden, M., Jensen, P. G., Larsen, K. G., Samusev, M., Srba, J., & Zhao, G. (2022). STOMPC: Stochastic Model-Predictive Control with Uppaal Stratego. In A. Bouajjani, L. Holík, & Z. Wu (Eds.), *International Symposium on Automated Technology for Verification and Analysis: Automated Technology for Verification and Analysis* (pp. 327-333). Springer. https://doi.org/10.1007/978-3-031-19992-9_21

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal -

Take down policy

If you believe that this document breaches copyright please contact us at vbn@aub.aau.dk providing details, and we will remove access to the work immediately and investigate your claim.

STOMPC: Stochastic Model-Predictive Control with UPPAAL STRATEGO [★]

Martijn A. Goorden¹^[0000–0002–0641–7240], Peter G.
Jensen¹^[0000–0002–9320–9991], Kim G. Larsen¹, Mihhail Samusev^{1,2}, Jiří Srba¹,
and Guohan Zhao²

¹ Department of Computer Science, Aalborg University, Aalborg, Denmark
`{mgoorden, pgj, kgl, srba}@cs.aau.dk`

² Department of the Built Environment, Aalborg University, Aalborg, Denmark
`{msam, guohanz}@build.aau.dk`

Abstract. We present the new co-simulation and synthesis integrated-framework STOMPC for stochastic model-predictive control (MPC) with UPPAAL STRATEGO. The framework allows users to easily set up MPC designs, a widely accepted method for designing software controllers in industry, with UPPAAL STRATEGO as the controller synthesis engine, which provides a powerful tool to synthesize safe and optimal strategies for hybrid stochastic systems. STOMPC provides the user freedom to connect it to external simulators, making the framework applicable across multiple domains.

1 Introduction

Controller software has become increasingly dominant in cyber-physical systems. Functionality that previously was implemented by hardware is now being shifted towards software. Often cyber-physical systems are safety-critical, hence strong safety-related requirements are formulated for them. At the same time, quality objectives need to be considered, such as being as fast as possible or minimizing resource usage. Designing safe and optimal controller software manually is a challenge, and several formal methods have been developed to synthesize controller strategies automatically [1, 14, 15].

For stochastic hybrid systems, the tool UPPAAL STRATEGO [5, 10] is the newly emerged branch of the leading tool UPPAAL that can automatically synthesize safe and near-optimal controller strategies. It combines statistical model checking, synthesis for timed games, and reinforcement learning. UPPAAL STRATEGO has been applied successfully to several case studies [3, 6, 8, 11, 12].

Within industry, model predictive control (MPC) is a widely adopted method for designing controllers [7]. MPC schemes are popular as they yield high-performing control systems without expert intervention over long periods of time. This is achieved by periodically using a model to predict the system's

[★] This work is partly supported by the Villum Synergy project CLAIRE and the ERC Advanced Grant LASSO.

future behavior and calculate an optimal control strategy for the next time-bounded period [4]. Therefore, MPC schemes are also called *online control*, as they can adapt control strategies while the system is running.

UPPAAL STRATEGO conceptually fits well within MPC designs. Yet it lacks the ability to periodically update the model’s state and synthesize a new strategy. In previous work [11], bash scripts are created utilizing the command line interface of UPPAAL STRATEGO to do all the calculations periodically. Unfortunately, these bash scripts are very case specific and not well adaptable to other case studies. Furthermore, we noticed that for each new case study, researchers were repeatedly rediscovering MPC schemes for UPPAAL STRATEGO.

We present the co-simulation and synthesis integrated-framework STOMPC, which implements a basic MPC scheme using UPPAAL STRATEGO as the core engine for synthesizing the strategies. With this framework, we aim to greatly simplify the setup for different case studies by implementing standard functionalities for MPC schemes with UPPAAL STRATEGO in Python classes. Furthermore, STOMPC can be connected to external, domain specific, simulators (or in fact again UPPAAL STRATEGO) that represent the real world. This makes the framework applicable to cases from different domains. Our framework is accessible on GitHub³, can be installed through pip, and its documentation is available⁴. An artifact for evaluation can be downloaded from Zenodo⁵.

2 Framework Overview

MPC captures a particular way of designing controllers for a broad range of systems and processes. It has the following three characteristics [4]: a model, which is used to predict the future of the system within a certain horizon, the calculation of a control sequence (or strategy) that optimizes some objective, and a receding approach, where all calculations are repeated after executing the first control action from the sequence and observing the true state as a consequence of that.

Fig. 1 provides a conceptual overview of the key ingredients of MPC that are implemented by STOMPC. Up to time $t = k$, we have observed the true state of the system x and provided control input u to it. Using a model of the system, we can predict the future state $\hat{x}_k$ within the control horizon. The evolution of the state depends on the control sequence being applied $\hat{u}_k$, where the applied control action can be switched after each control period. To determine which control sequence to choose, the objective is optimized. Often the objective is to minimize the difference between the state of the system and a reference signal.

Once the optimal control sequence is obtained, the first control action of this sequence is applied. When the end of the control period is reached, the process mentioned above is repeated. At time $t = k + p$, where p is the duration of the control period, the true value of the state of the system $x(k + p)$ is observed,

³ <https://github.com/DEIS-Tools/strategoutil>

⁴ <https://strategoutil.readthedocs.io/en/latest/>

⁵ <https://doi.org/10.5281/zenodo.6519909>

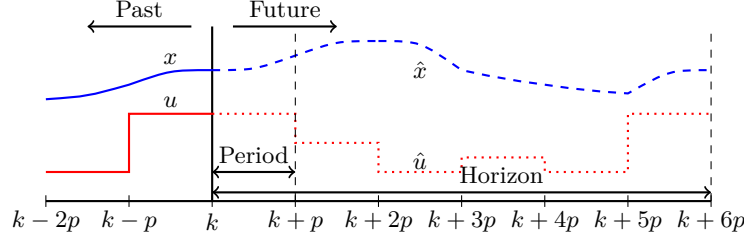


Fig. 1. Conceptual overview of model predictive control. In blue (dashed line) is the continuous evolution of the state in the past x and for the future $\hat{x}$, while red (dotted line) shows the periodically switched control signal in the past u and for the future $\hat{u}$.

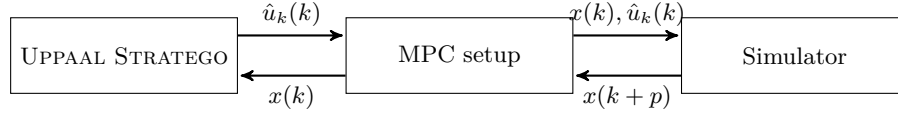


Fig. 2. Global architecture of STOMPC, where the MPC setup starts a new step at time $t = k$. After each step, k is replaced by $k + p$ and everything is repeated.

which, most likely, is different from the predicted state $\hat{x}_k(k + p)$. Repeating the calculation with the new true state $x(k + p)$ might result in a different control sequence $\hat{u}_{k+p}$ than the one calculated before $\hat{u}_k$.

STOMPC implements this MPC scheme using Python, hiding as much details as possible, such that a user can focus more on the application itself. Fig. 2 shows the architecture of STOMPC. It provides the component MPC setup, which orchestrates the MPC scheme. At time $t = k$ for some k , it supplies the current true state of the system $x(k)$ to UPPAAL STRATEGO. It does this by inserting the state values into the UPPAAL STRATEGO model. Subsequently, the MPC setup runs UPPAAL STRATEGO with this model to calculate the optimal control strategy. From the report generated by UPPAAL STRATEGO, the MPC setup identifies the calculated control action $\hat{u}_k(k)$ for the next control period.

After this, the MPC setup switches to the simulator. This simulator can be again UPPAAL STRATEGO or an external, domain specific one (see Section 3 for examples), or the actual physical system. The MPC setup supplies the simulator with the calculated control action $\hat{u}_k(k)$ for the next control period and, for memory-less simulators, also the last recorded true state $x(k)$ from which the simulator should continue. Subsequently, the simulator returns the true state $x(k + p)$ at the end of the control period. After that, the above procedure repeats until the end of the experiment.

More information on the setup of the tool, including a detailed example, can be found in the tool's documentation⁶.

⁶ <https://strategoutil.readthedocs.io>

3 Use Cases

An advantage of STOMPC is its general applicability across different application domains. We now discuss three use cases from different application domains: floorheating in a family house, storm water detention ponds, and traffic light control.

3.1 Floorheating in a Family House

The MPC scheme from Section 2 is in collaboration with the company Seluxit applied to controlling floor heating in a family house located in Northern Jutland, Denmark. Fig. 3 shows a screenshot of a digital twin of the house, displaying all its 10 rooms and the water pipes supplying heat to the rooms. Each room has its individually controlled target temperature (the upper digits in the rooms) and the thermodynamic equations used in the model consider the heat exchange between the rooms, between the rooms and their outside envelope, as well as the heat exchange from the water pipes passing under rooms.

In each 15 minute period, temperature sensors in each room report the current readings to the central control unit. During the following 15 minutes, the server gathers a 24-hour weather forecast and computes an optimal control strategy for the next 75 minutes using UPPAAL STRATEGO. The computed strategy optimizes the comfort in each room.

Simulations on the digital twin using the UPPAAL STRATEGO online controller (where the real house behavior is replaced by a Simulink model) show an average 40% improvement in comfort, compared to the controller that was used in the house before. As a side effect of the predictive control, the new UPPAAL STRATEGO control saves about 10% of energy. Further details about this concrete application of MPC can be found in [2, 11].



Fig. 3. Digital twin of a floor heating system

3.2 Stormwater Detention Ponds

Stormwater Detention Ponds are critical real-time control assets in urban stormwater management systems. They reduce the considerable hydraulic impact towards the natural stream, as well as avoid significant pollutant loads being discharged. However, only passive control of the stormwater pond outlet valves is currently used in Danish engineering practice.

We implement a co-simulation by combining UPPAAL STRATEGO with the domain specific simulator EPA-SWMM [9], as shown in Fig. 4. EPA-SWMM is an open-source physical-based dynamic rainfall-runoff model that has been implemented for decades in the urban stormwater management [9].

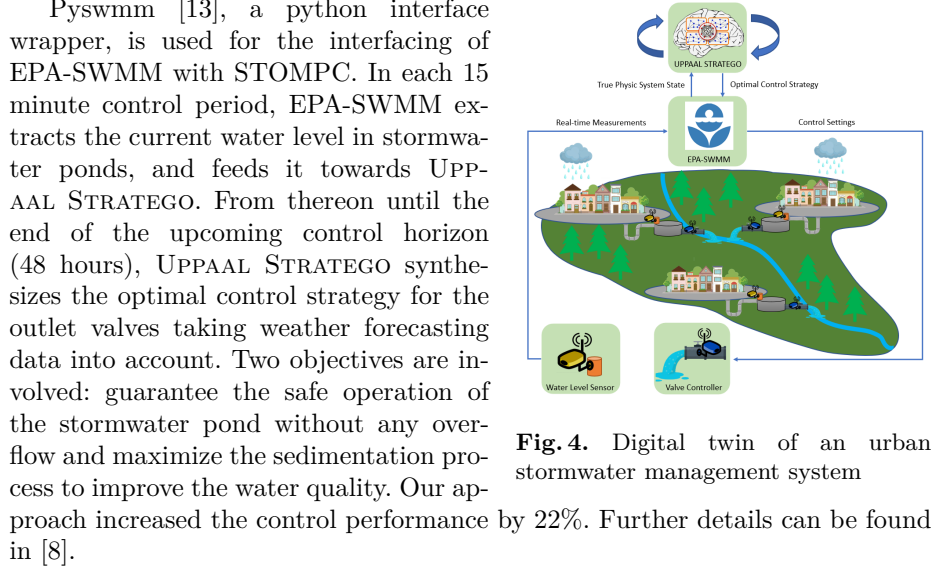


Fig. 4. Digital twin of an urban stormwater management system

3.3 Traffic Light Control

The application of MPC is widespread in the domain of traffic control. Recently UPPAAL STRATEGO has been successfully used to minimize the delays, queue lengths, number of stops, and fuel consumption of vehicles traveling on the arterial street Hobrovej in Aalborg simulated in VISSIM [6]. The street consists of 4 signalized intersections as shown in Fig. 5. The original traffic light controllers are pre-timed or detector time-gap based.

Every second UPPAAL STRATEGO is called to solve a traffic light configuration sequence planning problem that minimizes the total intersection delay. The vehicle information communicated to UPPAAL STRATEGO are the estimated times of arrival extracted from VISSIM's area sensors for each vehicle within 200m of the intersection. The first step in the resulting optimal control sequence is then sent back to VISSIM. Compared to the original control, and considering an intersection with smallest improvements, the described MPC approach manages to reduce the delays by 27%, queue lengths by 42%, number of stops by 20% and fuel consumption by 19%.



Fig. 5. Intersections optimized by UPPAAL STRATEGO at Hobrovej, Aalborg

In the original paper the data exchange between UPPAAL STRATEGO and VISSIM was established using a Python script. STOMPC can with minimal adjustments wrap the complexity of the communication between those two pieces

of software and let the user focus on the more high-level problems such as the definition of input data, objective function, and MPC parameters.

References

1. Abadi, M., Lamport, L., Wolper, P.: Realizable and unrealizable specifications of reactive systems. In: ICALP. pp. 1–17. Springer (1989)
2. Agesen, M., Larsen, K., Mikucionis, M., Muniz, M., Olsen, P., Pedersen, T., Srba, J., Skou, A.: Toolchain for user-centered intelligent floor heating control. In: IECON. pp. 5296–5301. IEEE (2016). <https://doi.org/10.1109/IECON.2016.7794040>
3. Ashok, P., Křetínský, J., Larsen, K.G., Le Coënt, A., Taankvist, J.H., Weininger, M.: SOS: Safe, optimal and small strategies for hybrid markov decision processes. In: Parker, D., Wolf, V. (eds.) QEST. pp. 147–164. LNCS (2019). https://doi.org/10.1007/978-3-030-30281-8_9
4. Camacho, E.F., Alba, C.B.: Model predictive control. Springer (2013)
5. David, A., Jensen, P.G., Larsen, K.G., Mikučionis, M., Taankvist, J.H.: Uppaal stratego. In: Baier, C., Tinelli, C. (eds.) TACAS. pp. 206–211. LNCS (2015). https://doi.org/10.1007/978-3-662-46681-0_16
6. Eriksen, A., Lahrmann, H., Larsen, K., Taankvist, J.: Controlling signalized intersections using machine learning. *Transportation Research Procedia* **48**, 987–997 (2020). <https://doi.org/10.1016/j.trpro.2020.08.127>
7. García, C.E., Prett, D.M., Morari, M.: Model predictive control: Theory and practice – a survey. *Automatica* **25**(3), 335–348 (1989). [https://doi.org/10.1016/0005-1098\(89\)90002-2](https://doi.org/10.1016/0005-1098(89)90002-2)
8. Goorden, M.A., Larsen, K.G., Nielsen, J.E., Nielsen, T.D., Rasmussen, M.R., Srba, J.: Learning safe and optimal control strategies for storm water detention ponds. *IFAC-PapersOnLine* **54**(5), 13–18 (2021). <https://doi.org/10.1016/j.ifacol.2021.08.467>
9. Huber, W.C., Rossman, L.A., Dickinson, R.E.: Epa storm water management model, swmm5. *Watershed models* **338**, 359 (2005)
10. Jaeger, M., Jensen, P.G., Larsen, K.G., Legay, A., Sedwards, S., Taankvist, J.H.: Teaching stratego to play ball: Optimal synthesis for continuous space MDPs. In: Chen, Y.F., Cheng, C.H., Esparza, J. (eds.) ATVA. pp. 81–97. LNCS (2019). https://doi.org/10.1007/978-3-030-31784-3_5
11. Larsen, K.G., Mikučioni, M., Muñiz, M., Srba, J., Taankvist, J.H.: Online and compositional learning of controllers with application to floor heating. In: Chechik, M., Raskin, J.F. (eds.) TACAS. pp. 244–259. LNCS (2016). https://doi.org/10.1007/978-3-662-49674-9_14
12. Larsen, K.G., Mikučioni, M., Taankvist, J.H.: Safe and optimal adaptive cruise control. In: Meyer, R., Platzer, A., Wehrheim, H. (eds.) *Olderog-Festschrift*, pp. 260–277. LNCS, Springer (2015). https://doi.org/10.1007/978-3-319-23506-6_17
13. McDonnell, B.E., Ratliff, K., Tryby, M.E., Wu, J.J.X., Mullapudi, A.: Pyswmm: The python interface to stormwater management model (swmm). *Journal of Open Source Software* **5**(52), 2292 (2020). <https://doi.org/10.21105/joss.02292>
14. Pnueli, A., Rosner, R.: On the synthesis of an asynchronous reactive module. In: ICALP. pp. 652–671. Springer (1989)
15. Ramadge, P.J., Wonham, W.M.: Supervisory control of a class of discrete event processes. *SIAM journal on control and optimization* **25**(1), 206–230 (1987)